

Q6) FAIRGAME: a Framework for AI Agents Bias Recognition using Game Theory

1. What is the role of Game theory in AI agent collaboration?

When several AI agents work together, the best choice for one agent depends on what the others choose. Game theory models this as a game, with players, the strategies they can pick, and a payoff for each combination of choices. From this we can predict what rational agents should do, for example whether they will cooperate or act selfishly.

In FAIRGAME, game theory is the reference point. LLM agents play classic games against each other, and their choices are compared with what game theory predicts (the Nash equilibrium). If the agents move away from that prediction because of the prompt language, the LLM used or the personality they were given, it shows a bias. So, game theory is used to understand how the agents make decisions, and to detect when their behaviour is inconsistent or unfair.

2. What is the importance of game theory?

- It gives a mathematical way to model and predict strategic decisions, for both humans and AI agents.
- It helps explain why agents behave the way they do, which makes multi-agent AI systems easier to interpret and trust.
- Many real uses of AI agents involve strategy, such as dispute resolution, auctions, pricing and supply chain negotiations. Hidden biases there can lead to unfair decisions, and game theory gives a baseline to check the agents against.
- It helps predict behaviour that only appears when agents interact over time, for example agents slowly learning to cooperate over repeated rounds.

3. What are the two classical Game theories presented in the paper? And how it can be used for the 5G applications?

The paper uses the Prisoner's Dilemma and the Battle of the Sexes.

Prisoner's Dilemma: two players each choose to cooperate or defect. Defecting is always better for each player on their own, so both end up defecting, which is the Nash equilibrium. But if both had cooperated, both would have been better off. In the paper's conventional version, both cooperating gives 2 years of penalty each, while both defecting gives 6 years each.

Battle of the Sexes: this is a coordination game. Both players want to choose the same option, but each prefers a different one. Both choosing A gives (10, 7), both choosing B gives (7, 10), and choosing differently gives (0, 0). Agreeing matters more than getting your preferred option.

Use in 5G: the paper does not discuss 5G, so these are my own examples.

- **Prisoner's Dilemma:** two neighbouring base stations (or two operators) share the same spectrum. Each can keep its transmit power low (cooperate) or increase it to give its own users better speeds (defect). If only one increases power, it gains. If both do, interference rises and both give worse service than if both had stayed low. Game theory can be used to design power control rules or incentives so that both stations are better off cooperating.
- **Battle of the Sexes:** in device-to-device (D2D) communication, two devices need to pick the same channel to connect. Each prefers a different channel, for example the one with less interference on its side, but if they pick different channels, the link fails and neither gets anything. Game theory helps design a rule, so the devices agree on one channel quickly.

4. Enlist selected LLMs models presented in a paper.

- Llama 3.1 405B (Meta, 405 billion parameters, open source)
- Mistral Large (Mistral AI, 123 billion parameters, open source)
- GPT-4o (OpenAI, parameters not disclosed, proprietary)
- Claude 3.5 Sonnet (Anthropic, parameters not disclosed, proprietary)

All four were accessed through their APIs, using the default settings recommended by each provider, and were tested from 10 to 15 February 2025.

5. Write 5-slides presentation of the paper.

<https://canva.link/fairgame-presentation-23k-0074>

Q6) A semi-independent policies training method with shared representation for heterogeneous multi-agents' reinforcement learning

1. How can you describe POMDP mathematically?

A POMDP is like an MDP, except the agent cannot see the full state of the environment. It only gets an observation of it. The paper uses the multi-agent version, called a Dec-POMDP, defined as the tuple:

$$G = (N, S, A, P, R, \Omega, O, n, \gamma)$$

$N = \{1, \dots, n\}$: the set of n agents

S : the set of global states of the environment, $s \in S$

$A = A_1 \times A_2 \times \dots \times A_n$: the joint action space, where A_i is the set of actions agent i can take

$P(s' | s, a)$: the probability of moving to state s' after joint action a is taken in state s , so

$$P: S \times A \times S \rightarrow [0,1]$$

$R(s, a)$: the reward function, which gives the reward for taking joint action a in state s . In the cooperative setting, all agents share it.

Ω : the set of observations, where Ω_i is the set of observations agent i can receive

$O(s, i)$: the observation function, which gives agent i its local observation from the state. The agent never sees s directly.

$\gamma \in [0,1)$: the discount factor, so rewards received later count less than rewards received now

The aim is to find policies that maximise the expected discounted return, $\mathbb{E}[\sum_t \gamma^t r_t]$. With only one agent ($n = 1$), this becomes a standard POMDP $(S, A, P, R, \Omega, O, \gamma)$.

2. What are the application of Hard and soft parameter sharing?

Both ideas come from multi-task learning, where one system has to handle several tasks.

Hard parameter sharing: the tasks share the bottom layers of one network, and each task has its own top layers and output layer.

Written as an equation:

$$y_1 = g_1(f(x)), \quad y_2 = g_2(f(x))$$

where f is the shared part and g_1, g_2 are the task-specific parts. Applications:

- Multi-task deep networks, where one model is trained for several related tasks and the shared layers learn features useful for all of them.
- Running several tasks on a device with limited memory, since sharing layers reduces storage, bandwidth and energy costs.
- In this paper, heterogeneous multi-agent RL. Different agent types (for example Marines, Marauders and the Medivac in StarCraft) share the base layers but keep their own policy layers. This lets them share experience while still learning different behaviour, and the paper found it mainly speeds up training.

Soft parameter sharing: each task keeps its own model and its own parameters, and the models are only loosely connected (usually by keeping their parameters similar). It is used when tasks are less related and each need more freedom, but it needs more parameters and more memory than hard sharing.

3. Explain and decode equation no 1.

$$L_e(\theta_e) = \mathbb{E}_{(o,a,r,o') \sim \mathcal{D}} \left[\sum_i \|f_o(z_{a_i}, o_i, a_{-i}) - o'_i\|_2^2 + \lambda_e \sum_i (f_r(z_{a_i}, o_i, a_{-i}) - r)^2 \right]$$

This is the loss function of the encoder-decoder (prediction) model that learns the shared representation. Minimising it trains the model to predict each agent's next observation and reward.

$L_e(\theta_e)$: the loss, where θ_e are the parameters of the encoder-decoder

$\mathbb{E}_{(o,a,r,o') \sim \mathcal{D}}$: the average over transitions (observation, action, reward, next observation) sampled from the replay buffer $\mathcal{D}$

$\sum_i$: the sum over all agents

z_{a_i} : the latent encoding of agent i . The encoder only gets the agent's identity (its type), so z can only hold information about what that type is like.

o_i : agent i 's current observation

a_{-i} : the action input to the decoder. The text says the decoder receives the agent's action, although a_{-i} is normally the notation for the other agents' actions.

f_o : the observation decoder, which predicts the next observation

o'_i : the actual next observation of agent i

$\|\cdot\|_2^2$: the squared L2 distance, so the first term is the error in predicting the next observation

f_r : the reward decoder, which predicts the reward. $(f_r(\dots) - r)^2$ is the error in predicting the reward.

λ_e : a scaling factor that sets how much the reward error counts compared to the observation error

The idea: if the model can predict what happens next for an agent using only its type encoding z , then z must have learned what makes that type different from the others. This z then becomes the shared base for all agents' policies. The model is trained before RL using random actions. It needs much less data than RL, and the same data can be reused later for training the policies.

4. Explain intuition behind figure no 4 presented in the paper.

Figure 4 shows how the total reward for each agent is built. Thick black lines show the data flow, thin black lines show rewards, and red lines show the losses used to create the intrinsic rewards.

Normally an agent learns only from the environment's reward (r_t). Here it also gets two extra rewards that come from the prediction model in Equation 1:

- Exploration reward, r^e (Eq. 2): if the prediction model makes a large error on a state, that state is new or rarely visited, so the agent is rewarded for reaching it. This works like curiosity and pushes agents to explore. It is weighted by the mixer weights p_i^m , and its weight is reduced during training once the environment has been explored.
- Diversity reward, r^d (Eq. 3): this compares how well an agent's own type representation predicts the outcome against the representations of other types. Agents are rewarded when agents of the same type have similar representations and different types have different ones, so the types stay distinguishable.

These are combined as (Eq. 4):

$$r_{\text{tot}} = r_t + \lambda_e r^e + \lambda_d r^d$$

So, the figure shows that each agent is pushed to do well in the game, to try unfamiliar states, and to keep agent types clearly separated in the representation. The paper's ablation study supports this. Removing r^e lowered performance on all tested maps and removing r^d mainly slowed down learning early in training on the heterogeneous maps.

5. Write 5-slides presentation of the paper.

<https://canva.link/stsr-presentation-23k-0074>